Algorithmes simples sur les arbres

La structure de données "Arbre" étant naturellement récursive, les algorithmes présentés seront majoritairement récursifs.

Hauteur et taille

On choisit la convention où la hauteur d'un arbre vide est fixée à -1 (*voir page* Wikipedia *à ce sujet*).

Remarque

Avec une telle convention, un arbre avec un seul nœud aura une hauteur nulle.

Algorithme 1

Hauteur
Entrée: la racine r d'un arbre binaire $\mathcal{A}$
Sortie: la hauteur de $\mathcal{A}$

Si r n'est pas vide:
    $h_G \longleftarrow$ Hauteur(fils gauche de r)
    $h_D \longleftarrow$ Hauteur(fils droit de r)
    Renvoyer $1 + \max(h_G, h_D)$
Sinon:
    Renvoyer −1

A faire

Donner une implémentation en python de cet algorithme.

La taille d'un arbre vide vaut 0. L'algorithme récursif du calcul de la taille d'un arbre binaire $\mathcal{A}$ est évident.

Algorithme 2

Taille
Entrée: la racine r d'un arbre binaire $\mathcal{A}$
Sortie: le nombre de nœuds dans $\mathcal{A}$

Si r n'est pas vide:
    $n_G \longleftarrow$ Taille(fils gauche de r)
    $n_D \longleftarrow$ Taille(fils droit de r)
    Renvoyer $1 + n_G + n_D$
Sinon:
    Renvoyer 0

A faire

Donner une implémentation en python de cet algorithme.

On admettra que les algorithmes `Hauteur` et `Taille` sont corrects, et ont une complexité $\mathcal{O}(n)$ en temps , où n est le nombre de nœuds de l'arbre $\mathcal{A}$.

Parcours d'un arbre en profondeur d'abord

Parcourir un arbre c'est passer par chacun de ses nœuds. Ici, on se contentera d'afficher la valeur de chaque nœud.

Dans quel ordre parcourir les nœuds d'un arbre? Il existe plusieurs possibilités basées sur le moment où on traite la racine:

- si on traite d'abord le sous arbre gauche, puis la racine, puis le sous arbre droit, l'ordre est dit **infixe**;
- si on traite d'abord la racine, puis le sous arbre gauche, puis le sous arbre droit, l'ordre est dit **préfixe**;
- si on traite d'abord le sous arbre gauche, puis le sous arbre droit, puis la racine, l'ordre est dit **postfixe** ou **suffixe**.

Parcours infixe

Algorithme 3

ParcoursInfixe
Entrée: racine r d'un arbre binaire $\mathcal{A}$
Sortie: affichage de tous les noeuds de $\mathcal{A}$

Si r n'est pas vide:
    ParcoursInfixe(fils gauche de r)
    Afficher la valeur (*clé*) de r
    ParcoursInfixe(fils droit de r)

L'algorithme **ParcoursInfixe** affiche les valeurs de tous les nœuds de $\mathcal{A}$, en temps $\mathcal{O}(n)$ où n est le nombre de nœuds dans $\mathcal{A}$.

A faire

Implémenter l'algorithme du parcours infixe d'un arbre, en python. Tester.

Parcours préfixe

Le parcours préfixe affiche d'abord la valeur de la racine.

Algorithme 4

ParcoursPrefixe
Entrée: racine r d'un arbre binaire $\mathcal{A}$
Sortie: affichage de tous les noeuds de $\mathcal{A}$

Si r n'est pas vide:
    Afficher la valeur (*clé*) de r
    ParcoursPrefixe(fils gauche de r)
    ParcoursPrefixe(fils droit de r)

Le coût en temps est évidemment identique au parcours infixe, c'est-à-dire en $\mathcal{O}(n)$ si n est la taille de l'arbre.

A faire

Implémenter l'algorithme du parcours préfixe d'un arbre, en python. Tester.

le parcours postfixe

L'affichage de la racine d'un arbre se fait **après** celui de son fils puis de son fils droit.

Algorithme 5

ParcoursPostfixe
Entrée: racine r d'un arbre binaire $\mathcal{A}$
Sortie: affichage de tous les noeuds de $\mathcal{A}$

Si r n'est pas vide:
    ParcoursPostfixe(fils gauche de r)
    ParcoursPostfixe(fils droit de r)
    Afficher la valeur (*clé*) de r

A faire

Implémenter l'algorithme du parcours postfixe d'un arbre, en python. Tester.

Parcours en largeur d'abord ou parcours par niveau

Il s'agit d'afficher tous les nœuds, niveau par niveau. On peut imaginer dans un premier temps utiliser une fonction auxiliaire `affiche_niveau` qui afficherait tous les nœuds situés à un niveau k (*avec* $0 \leq k \leq h$, *si* h *est la hauteur de l'arbre*). Puis, on bouclerait sur tous les niveaux. Voir implémentation ci-dessous.

```python
class Noeud:
    """ Une classe pour modéliser les noeuds d'un arbre"""

    def __init__(self, v, fg=None, fd=None):
        self.val = v
        self.gauche = fg
        self.droit = fd


def hauteur(arb):
    """ Renvoie la hauteur de l'arbre 'arb'"""

    if arb is None:
        return -1
    else:
        return 1 + max(hauteur(arb.gauche), hauteur(arb.droit))

def affiche_niveau(a, k):
    """ affiche tous les nœuds d'un arbre a, situés au niveau k"""
    if a is not None:
        if k == 0:
            print(a.val, end=' ')
        else:
            affiche_niveau(a.gauche, k - 1)
            affiche_niveau(a.droit, k - 1)

def parcours_niveau(a):
    h = hauteur(a)
    for i in range(h + 1):
        affiche_niveau(a, i)
```

```python
# TESTS
arb1 = Noeud('1',
             Noeud('2',
                   Noeud('4'),
                   Noeud('5')
                  ),
             Noeud('3',
                   Noeud('6',
                         Noeud('8',
                               Noeud('10'),
                               Noeud('11')
                              ),
                         Noeud('9',
                               Noeud('12'),
                               Noeud('13')
                              )
                        ),
                   Noeud('7')
                  )
             )
parcours_niveau(arb1)
```

```
1 2 3 4 5 6 7 8 9 10 11 12 13
```

Néanmoins cette solution n'est pas très efficace. Une analyse de l'algorithme permet de montrer qu'il a une complexité en temps en $\mathcal{O}(n^2)$ si n est la taille de l'arbre. Une solution *itérative* plus efficace consiste à utiliser une file pour retenir les nœuds à afficher dans le bon ordre.

Algorithme 6

ParcoursLargeur
Entrée : la racine r d'un arbre binaire $\mathcal{A}$
Sortie : affichage de tous les nœuds de $\mathcal{A}$, niveau par niveau et de gauche à droite

F $\longleftarrow$ file vide
Si r n'est pas vide:
    enfiler r dans F
Tant que F est non vide:
    $x \longleftarrow$ défiler un élément de F
    Afficher la valeur (*clé*) de x
    Si $\mathrm{fils_gauche}(x)$ n'est pas vide:
        enfiler $\mathrm{fils_gauche}(x)$ dans F
    Si $\mathrm{fils_droit}(x)$ n'est pas vide:
        enfiler $\mathrm{fils_droit}(x)$ dans F

A faire

Implémenter l'algorithme précédent en python. Tester. *Note*: on pourra utiliser la classe `File` déjà rencontrée dans les chapitres précédents ou utiliser la classe `Queue` du module `queue` de la librairie standard de python.

L'algorithme **ParcoursLargeur** affiche les valeurs de tous les nœuds de $\mathcal{A}$ dans l'ordre du parcours en largeur, en temps $\mathcal{O}(n)$ où n est la taille de $\mathcal{A}$.